

Instituto Tecnológico de Costa Rica

Escuela Ingeniería en Computadores

Curso: CE-1103

Algoritmos y Estructuras de Datos I

Proyecto 1

Monopoly Distribuido con Estructuras Lineales

Estudiantes:

María Celeste Escobar Morera

Carnet 2026099131

Abigail Saborio Jiménez

Carnet 2026097790

Andron Umaña López

Carnet 2026020123

Kendall Vargas Palma

Carnet 2026106335

Profesor:

Santiago Gamboa Ramírez

Semestre II

2026

Introducción

El presente proyecto consiste en el desarrollo de un sistema distribuido de Monopoly Electrónico adaptado con temáticas de Costa Rica, diseñado e implementado para el curso CE1103 - Algoritmos y Estructuras de Datos I del Instituto Tecnológico de Costa Rica. El propósito principal de la aplicación no consiste en reproducir de manera exhaustiva todas las reglas comerciales de un juego de mesa tradicional, sino en utilizar este contexto como medio para aplicar los principios de la Programación Orientada a Objetos (POO), el desarrollo de estructuras de datos lineales implementadas desde cero y el diseño de una arquitectura de sistemas distribuidos basada en el modelo cliente-servidor.

Como parte de los requerimientos del proyecto, se restringe el uso de bibliotecas de colecciones nativas con el objetivo de fortalecer la comprensión de los algoritmos y de la gestión de estructuras de datos. Por esta razón, se desarrollaron estructuras genéricas enlazadas de manera propia, entre las que se encuentran listas simples, listas circulares dobles, colas circulares y colas destinadas al manejo de cartas. Estas estructuras son utilizadas para representar diferentes elementos del sistema, como el tablero, la administración de los turnos, el inventario de propiedades de los jugadores y el mazo de eventos.

El alcance del proyecto comprende la integración de diferentes componentes que trabajan de manera conjunta. En primer lugar, se encuentra la lógica de negocio y el modelo de dominio, encargado de representar las reglas y elementos principales de la partida. El sistema está diseñado para soportar hasta cuatro jugadores y cuenta con un tablero temático compuesto por 32 casillas inspiradas en diferentes provincias de Costa Rica, entre ellas Cartago, Alajuela, Puntarenas y San José. Dentro de la dinámica del juego se incluyen funcionalidades como la compra de propiedades, el cobro de alquileres, la utilización de cartas de eventos, la administración de la bancarrota y el registro de un historial de transacciones que permite mantener un seguimiento de las operaciones realizadas durante la partida.

En segundo lugar, el proyecto implementa una arquitectura cliente-servidor sobre una red local. En esta arquitectura, el servidor funciona como la autoridad central encargada de mantener y administrar el estado de la partida. Este componente gestiona las cuentas de los jugadores, procesa los movimientos, valida las operaciones y controla las reglas de negocio.

Por su parte, las terminales cliente funcionan como interfaces independientes desde las cuales los jugadores interactúan con el sistema. La comunicación entre estos componentes se realiza mediante un protocolo propio basado en texto plano utilizando sockets TCP, permitiendo el intercambio de información entre los clientes y el servidor dentro de la red local.

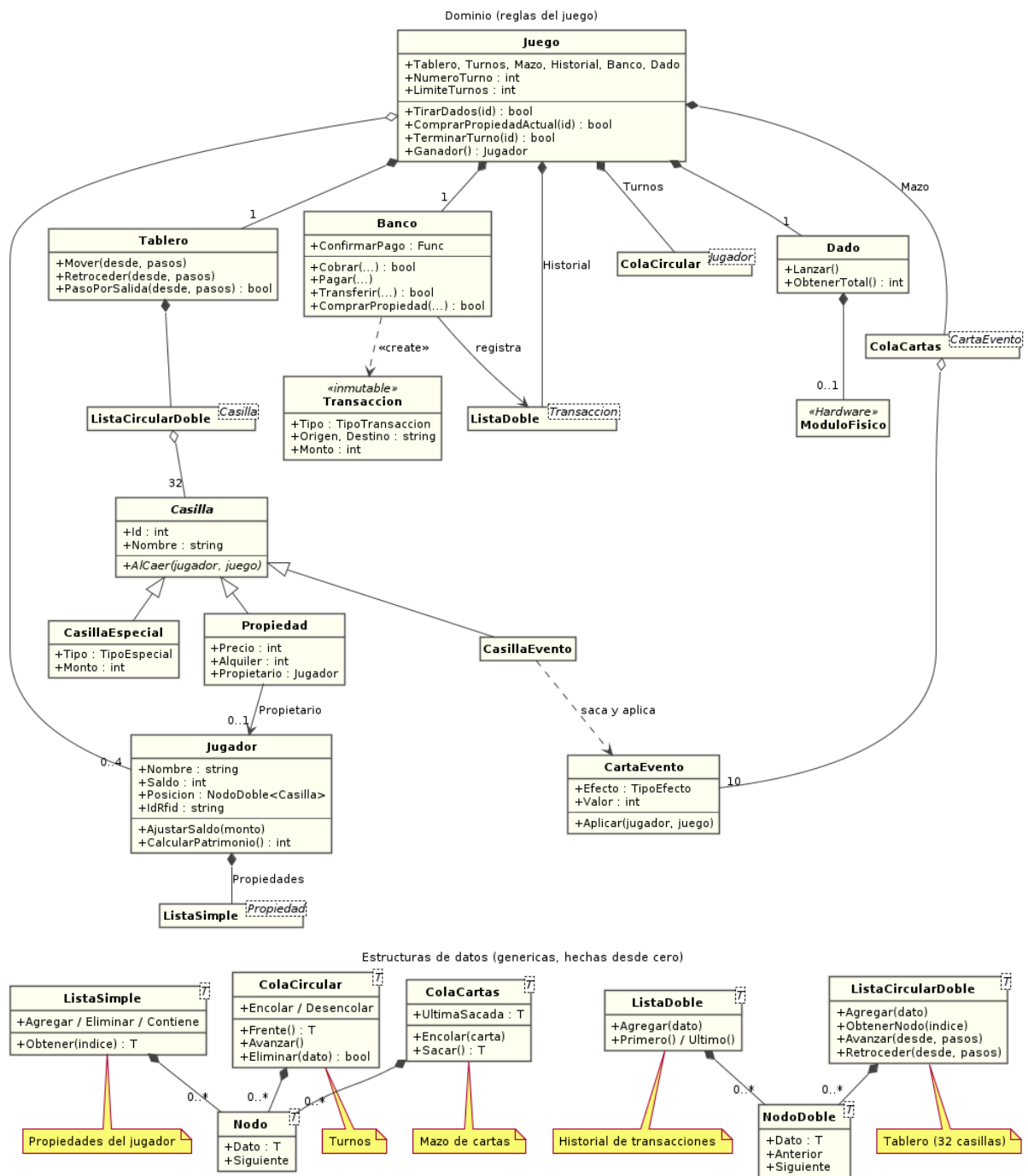
Finalmente, el proyecto incorpora un módulo electrónico de hardware basado en una Raspberry Pi Pico W, la cual se comunica directamente con el servidor mediante una conexión serial USB. Este módulo permite integrar elementos físicos al funcionamiento del juego, incluyendo un dado electrónico de dos dígitos implementado mediante displays de siete segmentos y activado por medio de un botón físico. Además, se incorpora un lector RFID RC522 que funciona como un mecanismo de identificación de los jugadores y permite realizar la autorización física de determinadas transacciones relacionadas con el banco.

La integración de estos componentes permite combinar el desarrollo de software, las estructuras de datos, las comunicaciones en red y el hardware en un único sistema. De esta manera, el proyecto permite aplicar los conocimientos adquiridos durante el curso en un entorno práctico, demostrando cómo el diseño orientado a objetos, la implementación de algoritmos y la organización estructurada de los componentes pueden utilizarse para desarrollar una aplicación distribuida que combina la interacción entre usuarios, dispositivos físicos y diferentes procesos de software.

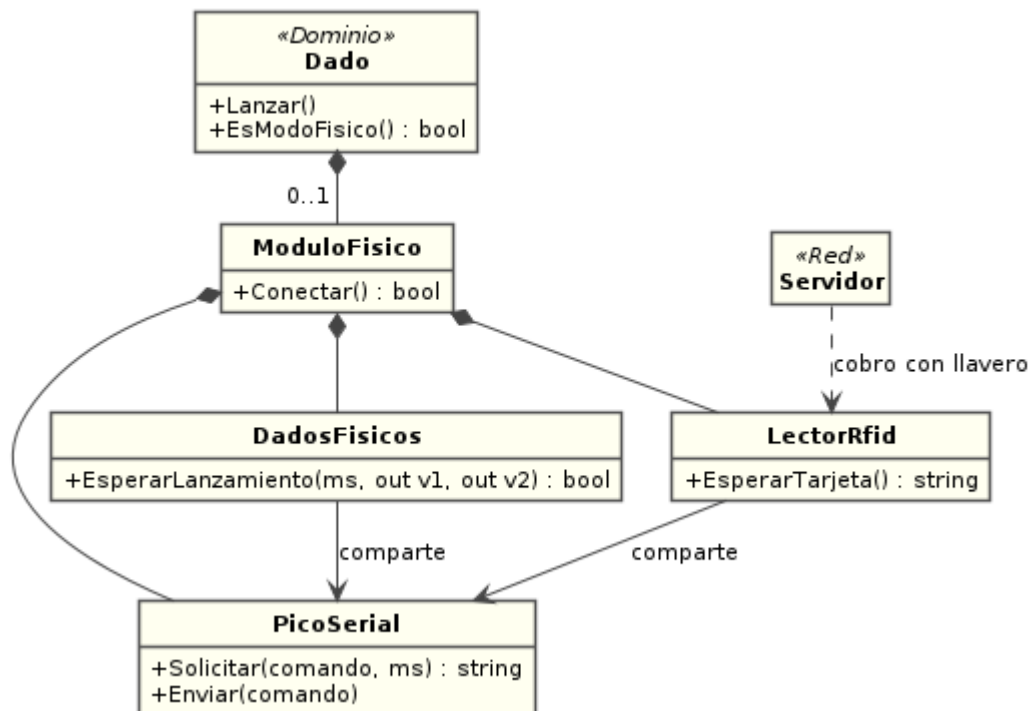
Diagramas UML

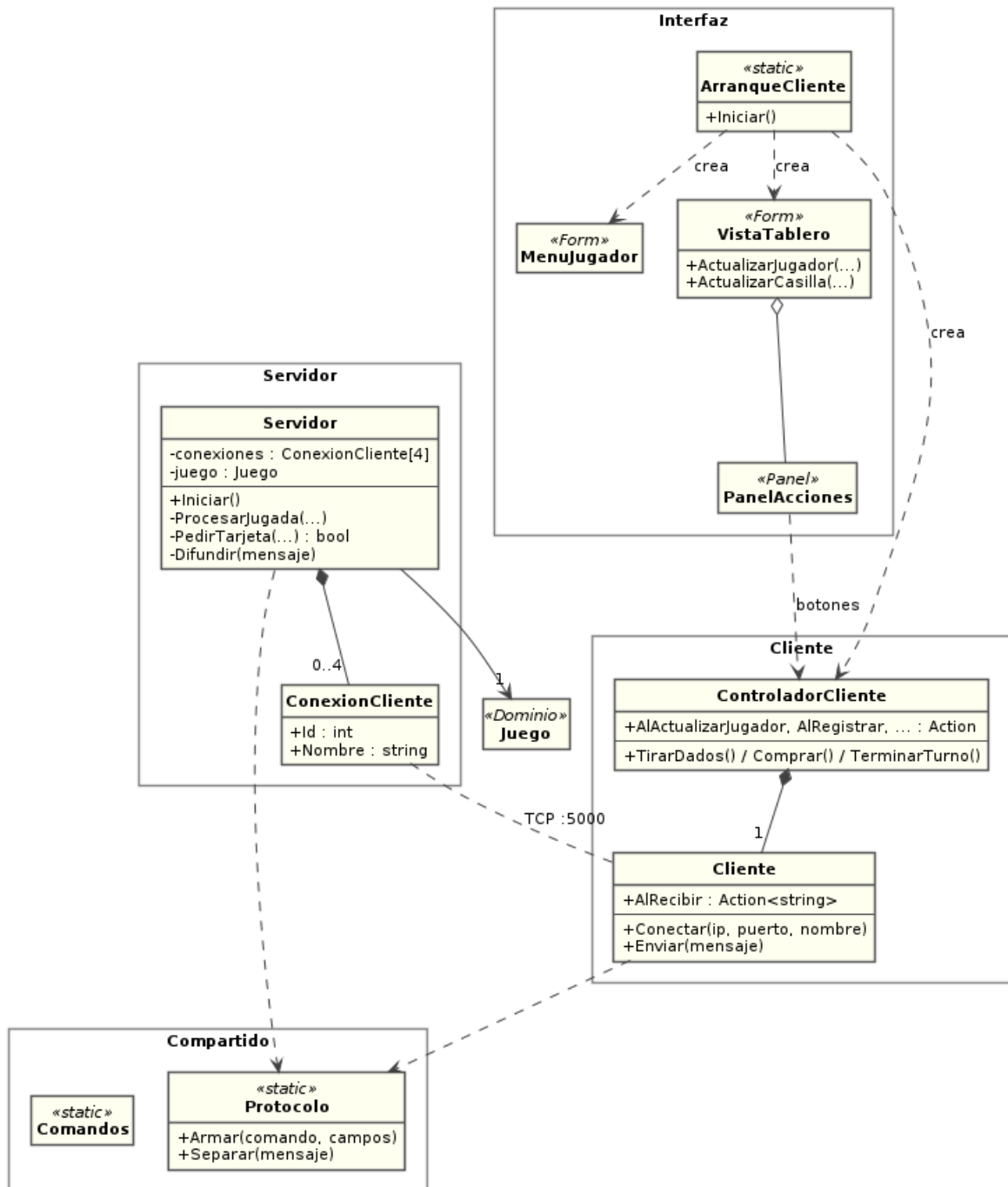
Para el diseño y modelado de la arquitectura del software se utilizaron diagramas UML, los cuales permiten representar visualmente la estructura y el funcionamiento del sistema. Estos diagramas facilitan la comprensión de la organización de las clases, las relaciones entre los diferentes componentes y la comunicación entre el cliente, el servidor y los dispositivos físicos utilizados en el proyecto.

1. **Diagrama de Clases (Dominio y Estructuras):** Este diagrama representa la estructura principal del sistema y las relaciones entre sus clases. Se muestra la jerarquía de las casillas del tablero, donde Propiedad, CasillaEvento y CasillaEspecial heredan de la clase abstracta Casilla. También se representa la clase Juego, encargada de administrar el estado general de la partida, y las diferentes estructuras de datos implementadas desde cero, como ListaSimple, ListaCircularDoble, ColaCircular, ColaCartas y ListaDoble, las cuales se mantienen separadas de la lógica principal del juego.
2. **Diagrama de Componentes y Red (Cliente-Servidor y Hardware):** Este diagrama representa la distribución de los componentes del sistema y la comunicación entre ellos. El servidor se encarga de administrar el estado de la partida, el Banco y la comunicación con la Raspberry Pi Pico W mediante conexión serial. Por otro lado, las terminales cliente se encargan principalmente de mostrar la interfaz gráfica y enviar o recibir información mediante sockets TCP. Esta separación permite mantener diferenciadas las responsabilidades del cliente, el servidor y los dispositivos físicos.



Hardware (dato fisico + lector RFID por USB)





Documentación Estructuras Utilizadas

1. Criterios generales

Todas las estructuras de datos utilizadas en el proyecto fueron implementadas desde cero, sin utilizar `System.Collections.Generic` ni estructuras nativas de C#, como `List`, `LinkedList`, `Queue`, `Stack` o `Dictionary`. Esta restricción se encuentra reforzada en el archivo `Monopoly.csproj` mediante la desactivación de los `ImplicitUsings`, por lo que cualquier intento de utilizar estas estructuras genera un error de compilación.

Debido a esta restricción, tampoco se utiliza `IEnumerable<T>` ni `foreach` para recorrer las estructuras propias. Los recorridos se realizan mediante ciclos `while`, utilizando una referencia que avanza de un nodo al siguiente. Aunque se prohíbe el uso de las estructuras de `System.Collections.Generic`, sí es válido utilizar genéricos en las estructuras desarrolladas por el grupo, ya que los genéricos forman parte del lenguaje C# y no de dicha biblioteca.

El profesor también autorizó el uso de arreglos nativos `T[]` como almacenamiento interno. Sin embargo, el grupo decidió implementar las estructuras mediante nodos enlazados, con el objetivo de mantener coherencia con los contenidos del curso. Además, las estructuras ubicadas en la carpeta `Estructuras/` son completamente independientes del dominio del juego, ya que no contienen referencias a clases como `Jugador`, `Casilla` o `Transaccion`. Esto permite reutilizarlas en diferentes contextos.

2. Nodos

Para construir las estructuras enlazadas se utilizan dos tipos principales de nodos. El primero es `Nodo<T>`, que contiene el dato almacenado y una referencia al siguiente nodo. Este tipo de nodo se utiliza en `ListaSimple`, `ColaCircular` y `ColaCartas`.

El segundo es `NodoDoble<T>`, que contiene el dato y dos referencias: una hacia el nodo anterior y otra hacia el siguiente. Este nodo es utilizado por `ListaCircularDoble` y `ListaDoble`. En ambos casos, el constructor crea un nodo independiente que todavía no se encuentra enlazado con otros nodos.

3. ListaSimple — Propiedades de cada jugador

La `ListaSimple<T>` se utiliza para almacenar las propiedades que cada jugador adquiere durante la partida. Cuando un jugador compra una propiedad, esta se agrega a su lista. Posteriormente, la lista puede recorrerse para calcular el patrimonio final del jugador.

Esta estructura se eligió porque las propiedades no necesitan recorrerse en sentido inverso y las operaciones principales son agregar elementos y comprobar si una propiedad pertenece al jugador. La lista mantiene una referencia a su cabeza y un contador de elementos.

Entre las operaciones disponibles se encuentran agregar, eliminar, comprobar si un elemento existe, obtener un elemento mediante su índice, contar los elementos y verificar si la lista está vacía.

4. ListaCircularDoble — Tablero

La ListaCircularDoble<T> representa el tablero del juego. Cada nodo corresponde a una casilla y mantiene referencias tanto a la casilla anterior como a la siguiente. Aunque el enunciado establece un mínimo de 24 casillas, el grupo implementó un tablero de 32 casillas, distribuidas en cuatro lados de ocho casillas y con una temática relacionada con las provincias de Costa Rica.

La estructura circular permite que la última casilla se conecte nuevamente con la primera. De esta manera, el movimiento por el tablero se realiza recorriendo realmente los nodos de la estructura, sin necesidad de calcular directamente la posición mediante índices.

El enlace doble también permite desplazarse hacia atrás. Por ejemplo, cuando un jugador utiliza la carta que indica retroceder dos casillas, el método Retroceder recorre los nodos mediante la referencia Anterior. De esta forma, si el jugador se encuentra en la Salida, retroceder dos casillas permite continuar naturalmente desde el final del círculo.

La estructura también permite detectar cuando un jugador pasa por la Salida. Durante el recorrido se revisan las casillas intermedias y la casilla final para determinar si el jugador cruzó por la posición correspondiente a la Salida.

Dentro del dominio, la clase Tablero utiliza esta lista para construir y administrar las 32 casillas, además de proporcionar operaciones como mover, retroceder, obtener una casilla y comprobar el paso por la Salida.

5. ColaCircular — Turnos de los jugadores

La ColaCircular<T> se utiliza para administrar el orden de los turnos. El jugador ubicado al frente de la cola es quien tiene el turno actual. Al finalizar su turno, la estructura avanza hacia el siguiente jugador.

La naturaleza circular de la cola permite representar directamente el comportamiento de los turnos, ya que después del último jugador vuelve a participar el primero. Cuando un jugador es eliminado por no poder cubrir un pago, se utiliza la operación Eliminar para retirarlo de la rotación.

Es importante distinguir entre Avanzar y Desencolar. Avanzar cambia el jugador que tiene el turno sin eliminar a nadie, mientras que Desencolar extrae al elemento que se encuentra al frente. En el funcionamiento normal del juego se utiliza principalmente Avanzar.

La cola mantiene referencias al frente y al último nodo, además de un contador de elementos. También se realizan pruebas para comprobar su funcionamiento cuando está vacía, cuando contiene un único elemento y cuando se eliminan jugadores ubicados en diferentes posiciones.

6. ColaCartas — Mazo de cartas de evento

La ColaCartas<T> administra el mazo de cartas de evento. Cuando un jugador cae en una casilla de evento, se obtiene la carta que está al frente, se aplica su efecto y posteriormente la carta se coloca nuevamente al final del mazo.

Esta estructura se utiliza porque su funcionamiento coincide con la regla establecida para las cartas del juego: una carta utilizada debe regresar al final para poder utilizarse nuevamente más adelante.

El método Sacar realiza todo este proceso de forma automática. Toma la carta del frente, mueve el frente hacia la siguiente carta y coloca la carta utilizada al final. Por lo tanto, la cantidad de cartas del mazo no cambia.

El mazo comienza con diez cartas definidas por el juego y estas se van reutilizando conforme los jugadores caen en las casillas de evento. Además, la propiedad UltimaSacada permite conocer cuál fue la última carta obtenida para poder comunicar su resultado a los jugadores.

7. ListaDoble — Historial de transacciones

La ListaDoble<T> almacena el historial de todas las transacciones económicas generadas durante la partida. Entre estas operaciones se encuentran las compras de propiedades, pagos de alquiler, pagos al banco, transferencias entre jugadores, ganancias o pérdidas por eventos y premios obtenidos al pasar por la Salida.

Esta estructura se utiliza porque el historial debe poder recorrerse tanto desde la transacción más antigua hasta la más reciente como en sentido contrario. Al mantener referencias a la cabeza y a la cola, la lista doblemente enlazada permite realizar ambos recorridos sin necesidad de invertir la estructura.

El Banco es el responsable de registrar las transacciones. Cada operación económica genera una Transaccion, que posteriormente se agrega al historial. Las búsquedas por jugador o por tipo de transacción se realizan en la capa de dominio, de manera que la estructura de datos no necesita conocer detalles específicos del juego.

8. Costo de las operaciones

Las diferentes estructuras presentan distintos costos dependiendo de la operación realizada. Algunas operaciones pueden ejecutarse directamente porque la estructura mantiene referencias a los nodos necesarios, mientras que otras requieren recorrer los elementos uno por uno.

En ListaSimple, operaciones como Agregar, Eliminar, Contiene y Obtener requieren recorrer la estructura en determinadas situaciones. En el caso de Agregar, se debe llegar hasta el último nodo debido a que la lista no mantiene una referencia directa a la cola.

Por otro lado, ListaCircularDoble puede agregar elementos sin recorrer toda la estructura porque la referencia cabeza.Anterior permite acceder directamente al último nodo. Las operaciones Avanzar y Retroceder sí recorren tantos nodos como pasos se indiquen.

En ColaCircular, Encolar, Desencolar y Avanzar se realizan mediante referencias directas, mientras que Eliminar puede requerir un recorrido para localizar al jugador. En ColaCartas, tanto Encolar como Sacar se realizan directamente gracias a las referencias al frente y al final.

Finalmente, ListaDoble puede agregar elementos directamente al final porque mantiene una referencia a la cola. Los recorridos completos de esta lista requieren avanzar nodo por nodo, ya sea desde la cabeza o desde la cola.

9. Estructuras que no forman parte de las estructuras de datos del juego

Es importante diferenciar las estructuras utilizadas por la lógica del juego de aquellas que pertenecen únicamente a la interfaz gráfica. Por ejemplo, VistaTablero.cs utiliza un arreglo Point[] para almacenar las coordenadas donde deben dibujarse las fichas de los jugadores sobre la imagen del tablero.

Este arreglo no representa el tablero real. El tablero está implementado mediante la ListaCircularDoble<Casilla> y es administrado por la clase Tablero. El arreglo de coordenadas únicamente indica a la interfaz en qué posición de la pantalla debe dibujarse cada ficha.

La lógica del juego tampoco depende de este arreglo. El movimiento de los jugadores, los propietarios, los alquileres y los efectos de las casillas son manejados por el servidor mediante las estructuras enlazadas. Por esta razón, incluso si se eliminara la interfaz gráfica, la lógica principal del juego podría continuar funcionando.

Los demás arreglos utilizados tienen funciones específicas y limitadas, como almacenar temporalmente los jugadores, las conexiones, los nombres del menú o los datos recibidos para la representación gráfica. Estos arreglos no sustituyen ninguna de las estructuras de datos exigidas para el funcionamiento del juego.

10. Estado de implementación

Las siete estructuras principales se encuentran implementadas, probadas y utilizadas dentro del juego. Nodo.cs y NodoDoble.cs están completos y sirven como base para las estructuras enlazadas.

ListaSimple.cs cuenta con sus operaciones principales y ha sido probada mediante inserción, búsqueda, obtención y eliminación. ListaCircularDoble.cs se encuentra implementada con las operaciones de avance y retroceso utilizadas por el tablero de 32 casillas.

ListaDoble.cs permite recorrer el historial en ambos sentidos. ColaCircular.cs ha sido probada con diferentes casos de eliminación y con el avance completo de los turnos. Por su parte, ColaCartas.cs implementa el reciclaje de las cartas mediante la operación Sacar y mantiene la propiedad UltimaSacada.

Como regla del grupo, una vez que se establece una firma pública para una estructura, esta no debe modificarse sin comunicarlo previamente a los demás integrantes, ya que podría afectar el código que otros miembros ya hayan desarrollado.

11. Resumen: dónde se utiliza cada estructura

La ListaCircularDoble<Casilla> se encuentra en el Tablero y representa las 32 casillas del juego, permitiendo el movimiento hacia adelante y hacia atrás. La posición de cada jugador se representa mediante un NodoDoble<Casilla>, por lo que se mantiene una referencia directa al nodo correspondiente.

La ColaCircular<Jugador> se utiliza para administrar los turnos, incluyendo el salto de turnos perdidos y la eliminación de jugadores que quedan fuera de la partida. La ColaCartas<CartaEvento> representa el mazo de diez cartas que se reciclan después de ser utilizadas.

La ListaDoble<Transaccion> almacena el historial económico de la partida y permite recorrerlo en ambos sentidos. La ListaSimple<Propiedad> almacena las propiedades de cada jugador, mientras que otra ListaSimple<Transaccion> se utiliza para devolver los resultados de las búsquedas realizadas sobre el historial.

Todas estas estructuras se encuentran en el servidor. El cliente únicamente recibe información mediante la red y se encarga de representarla visualmente.

12. Pruebas

Las estructuras se pueden probar mediante el comando `dotnet run -- pruebas`, que ejecuta las pruebas no interactivas y muestra los resultados obtenidos.

Las pruebas de ListaDoble verifican la inserción, el conteo y los recorridos en ambas direcciones. Las pruebas de ListaSimple comprueban operaciones como agregar, buscar, obtener y eliminar elementos en diferentes posiciones.

Para ColaCircular se prueban casos como una cola vacía, un solo elemento, un giro completo y la eliminación de elementos ubicados al frente, en el medio, al final o como único elemento. En ColaCartas se verifica que las cartas puedan agregarse y que, al utilizar Sacar, regresen al final del mazo sin modificar la cantidad de elementos.

Finalmente, la ListaCircularDoble se prueba indirectamente mediante una partida completa, comprobando que el tablero mantenga correctamente su forma circular, que se otorgue el premio al pasar por la Salida y que funcionen las cartas de evento. También se prueban las búsquedas y los reportes del historial de transacciones en ambos sentidos.

Protocolo cliente-servidor

El proyecto Monopoly Distribuido con Estructuras Lineales, desarrollado para el curso CE1103 Algoritmos y Estructuras de Datos I del ITCR, utiliza una arquitectura cliente-servidor para permitir la comunicación entre los jugadores y el servidor central. La comunicación se realiza mediante sockets TCP sobre una red local, utilizando el puerto 5000.

Los mensajes se manejan como texto plano, donde cada mensaje corresponde a una línea y termina con un salto de línea. Los diferentes campos de cada mensaje se separan mediante el carácter `|`. La comunicación utiliza UTF-8 sin BOM, con el objetivo de evitar que caracteres adicionales interfieran con la interpretación de los comandos. El sistema está diseñado para trabajar con cuatro jugadores y permite un máximo de cuatro conexiones activas.

La comunicación se implementa principalmente mediante las clases *Protocolo*, *Comandos*, *Servidor*, *ConexionCliente*, *Cliente* y *ControladorCliente*. Se eligió un protocolo de texto debido a que facilita la depuración y permite observar fácilmente los mensajes que se intercambian entre el cliente y el servidor.

Principios de diseño

El servidor es la única autoridad sobre el estado del juego. Toda la información relacionada con el tablero, jugadores, saldos, turnos e historial se mantiene únicamente en el servidor. Los clientes solamente envían solicitudes y muestran la información que reciben, por lo que no realizan cálculos ni toman decisiones sobre el estado de la partida.

Cada conexión recibe un identificador de jugador entre 1 y 4, asignado por el servidor. El cliente no puede indicar qué número de jugador desea utilizar. Esto permite que el servidor determine quién realiza cada acción a partir de la conexión desde la cual recibe el mensaje.

Cada jugador es atendido mediante su propia conexión, pero las operaciones que modifican el estado del juego se procesan de forma controlada utilizando un único lock. De esta manera, se evita que dos jugadores modifiquen simultáneamente la información de la partida.

Además, todas las acciones son validadas por el servidor. Las restricciones visuales de la interfaz solamente facilitan el uso del programa y no representan una medida de seguridad.

Construcción y lectura de mensajes

La clase Protocolo se encarga de construir y analizar los mensajes enviados entre el cliente y el servidor. Los mensajes tienen una estructura sencilla basada en comandos y campos separados por `|`.

Por ejemplo, un mensaje como `DADOS|2|4` indica el resultado de un lanzamiento, mientras que `TIRAR_DADOS` representa una solicitud sin campos adicionales.

Antes de construir un mensaje, los campos se limpian para evitar que un nombre u otro dato contenga caracteres que puedan confundirse con separadores o saltos de línea. Los comandos se utilizan mediante las constantes definidas en `Comandos.cs`, evitando escribirlos directamente en diferentes partes del código.

Mensajes del cliente al servidor

El cliente puede enviar diferentes solicitudes dependiendo de la acción que el jugador quiera realizar. Al conectarse, envía `CONECTAR` junto con el nombre del jugador. Durante la partida puede solicitar acciones como `TIRAR_DADOS`, `COMPRAR_PROPIEDAD`, `NO_COMPRAR` y `TERMINAR_TURNO`.

También existen solicitudes de consulta, como `CONSULTAR_ESTADO` y `CONSULTAR_TRANSACCIONES`, que permiten obtener información de la partida sin modificar su estado.

Antes de aceptar una acción, el servidor verifica que el jugador esté conectado correctamente, que la partida haya comenzado, que no haya terminado y que corresponda su turno. Las consultas de estado y transacciones pueden realizarse independientemente del turno.

Mensajes del servidor al cliente

El servidor responde a las solicitudes mediante mensajes específicos. Al conectarse, el jugador recibe BIENVENIDO con el número de jugador asignado. Cuando se conectan los cuatro participantes, se envía INICIO y posteriormente se informa el turno mediante TURNO.

Durante una jugada, el servidor puede enviar mensajes como DADOS, MOVIMIENTO, CASILLA y OFRECER_COMPRA, dependiendo de las acciones realizadas. También se utiliza JUGADOR para informar el estado de los participantes y MENSAJE para mostrar avisos generales.

Cuando se consultan las transacciones, el servidor envía mensajes TRANSACCION con cada registro correspondiente. Finalmente, cuando la partida termina, se envía FIN indicando el nombre del ganador.

La partida comienza automáticamente cuando se conecta el cuarto jugador. No existe un mensaje independiente de INICIADO, ya que el botón de inicio del cliente solamente cambia la ventana de esa computadora.

Flujo de una partida

Al establecerse una conexión, el servidor asigna el primer espacio disponible al jugador. Después de recibir el mensaje CONECTAR, crea e incorpora al jugador al estado de la partida y comunica su identificador al cliente.

El orden de conexión determina el orden inicial de los turnos. Cuando se conecta el cuarto jugador, la partida comienza y el servidor comunica a todos los clientes el estado inicial y el jugador que debe comenzar.

Durante el turno, el jugador puede lanzar los dados. Si se utiliza la Raspberry Pi Pico, el servidor solicita la interacción con el dado físico y espera hasta 15 segundos por la pulsación del botón. Una vez obtenido el resultado, se informa a todos los clientes, se actualiza la posición del jugador y se muestran los datos de la casilla correspondiente.

Si el jugador cae en una propiedad disponible, el servidor le permite decidir si desea comprarla. Después de realizar la acción correspondiente, se actualizan los saldos, las posiciones y las transacciones de todos los jugadores.

Al finalizar una acción válida, el servidor comunica los cambios realizados y determina si la partida continúa, cambia de turno o ha llegado a su final.

Mensajes informativos y transacciones

El mensaje MENSAJE se utiliza para comunicar información general a los jugadores. Entre estos avisos se encuentran las conexiones y desconexiones, las transacciones realizadas, la aparición de cartas, la compra o no compra de propiedades, la eliminación de un jugador y las interacciones con el hardware.

Las transacciones se muestran con información sobre el origen, destino, monto y descripción de la operación. Además, el sistema aprovecha la estructura de datos de lista doblemente enlazada para recorrer el historial desde la última transacción registrada y obtener las nuevas operaciones que deben notificarse.

Sistema RFID

El lector RFID se utiliza para identificar al jugador durante determinadas operaciones, principalmente para confirmar compras o pagos. La tarjeta o llavero únicamente sirve como mecanismo de identificación; el saldo oficial y todas las operaciones económicas permanecen bajo control del servidor.

Para comprar una propiedad, el servidor puede solicitar la identificación mediante el llavero correspondiente. Si no se confirma correctamente, la compra se cancela. En el caso de pagos obligatorios, el sistema puede completar la operación según las reglas establecidas para evitar que una partida quede bloqueada.

Manejo de errores

Cuando una acción no cumple con las reglas del juego, el servidor responde utilizando el mensaje ERROR seguido del motivo correspondiente. Estos mensajes se envían únicamente al jugador que realizó la solicitud.

Entre los errores contemplados se encuentran intentar jugar fuera de turno, lanzar los dados más de una vez, intentar comprar una propiedad que ya tiene dueño, no tener suficiente saldo, intentar terminar el turno sin lanzar los dados o enviar un comando desconocido.

De esta manera, las reglas importantes del juego siempre se validan en el servidor, independientemente de lo que muestre o permita la interfaz del cliente.

Finalización y desconexiones

La partida termina cuando queda un solo jugador activo o cuando se alcanza el límite de 100 turnos individuales establecido en `Program.LimiteTurnos`. El ganador se determina entre los jugadores activos según su patrimonio, considerando el saldo y el valor de sus propiedades.

Al finalizar, el servidor envía el mensaje FIN a todos los clientes y genera el reporte de transacciones en `docs/transacciones/reporte_partida.txt`. Después de finalizar la partida, cualquier nueva acción será rechazada.

Si un cliente se desconecta, el servidor libera su conexión y notifica la situación a los demás jugadores. Actualmente existe la limitación de que, si un jugador se desconecta durante la partida, este puede permanecer dentro del orden de turnos y provocar que el sistema espere cuando llegue su turno.

Funcionamiento del cliente

El cliente recibe los mensajes mediante la clase `Cliente` y los procesa a través de `ControladorCliente`. Esta clase interpreta cada mensaje y comunica los cambios a la interfaz mediante eventos o acciones.

Por ejemplo, los mensajes JUGADOR actualizan la posición y el saldo mostrado, CASILLA actualiza la información de la última casilla, BIENVENIDO permite conocer el identificador asignado, INICIO indica que comenzó la partida y TURNO actualiza la información del jugador que debe jugar.

Debido a que los mensajes de red se reciben desde un hilo independiente, las ventanas utilizan `Invoke` para actualizar correctamente los controles de la interfaz gráfica.

Protocolo entre la PC y la Raspberry Pi Pico W

La comunicación entre la computadora y la Raspberry Pi Pico W es independiente del protocolo TCP utilizado entre clientes y servidor. La Pico no funciona como cliente TCP, sino que se conecta mediante USB a la computadora que ejecuta el servidor.

La comunicación utiliza el puerto serie virtual USB-CDC, a una velocidad de 115200 baudios, y también utiliza mensajes de texto. En este caso, los datos emplean : y , como separadores.

La clase Hardware/PicoSerial.cs es la encargada de controlar el puerto COM. Tanto el dado electrónico como el lector RFID utilizan el mismo puerto, por lo que esta clase mantiene el control exclusivo de la comunicación y distribuye los mensajes recibidos.

Entre los principales mensajes se encuentran PING, utilizado para comprobar la conexión con la Pico; MOSTRAR, utilizado para mostrar los valores de los dados; APAGAR, para apagar los displays; y READID, para solicitar la lectura de una tarjeta RFID.

La Pico puede responder con mensajes como DADOS:LISTO, DADOS:d1,d2,total, confirmaciones de los displays, el UID de una tarjeta RFID, TIMEOUT cuando no se detecta ninguna tarjeta o mensajes de error.

De esta manera, el protocolo permite integrar el hardware físico con el juego sin modificar la arquitectura cliente-servidor. La Raspberry Pi Pico funciona únicamente como un medio de entrada y salida, mientras que el servidor mantiene el control del estado y de las reglas de la partida.

Manual breve de ejecución

1. Requisitos

Para ejecutar el proyecto, los cuatro integrantes deben utilizar Windows, debido a que la aplicación utiliza Windows Forms. También es necesario tener instalado .NET 10 y el paquete *System.IO.Ports*, utilizado para la comunicación con la Raspberry Pi Pico.

Después de instalar el paquete, se debe subir al repositorio el archivo *Monopoly.csproj* actualizado, para que todos trabajen con la misma configuración.

2. Compilación

La compilación debe realizarse desde la raíz del repositorio. Esto es importante porque el proyecto utiliza rutas relativas para acceder a recursos como la imagen del tablero y las imágenes de las fichas. Si el programa se ejecuta desde otra carpeta, estos recursos podrían no encontrarse y generar errores.

3. Roles

El archivo *Program.cs* determina el modo de ejecución del programa. El proyecto cuenta con diferentes roles y modos:

- Servidor: administra la partida, la comunicación con los clientes y, opcionalmente, la Raspberry Pi Pico.
- Cliente gráfico: proporciona la interfaz visual del juego.
- Cliente de consola: permite jugar mediante comandos de texto y funciona como alternativa al cliente gráfico.
- Pruebas: permite comprobar diferentes componentes del sistema.
- Hardware: permite realizar pruebas relacionadas con la Raspberry Pi Pico.
- Tablero: permite comprobar visualmente el funcionamiento de VistaTablero con jugadores simulados.
-

4. Jugar en una sola PC

Para realizar una partida en una sola computadora, primero se debe compilar el proyecto. Después, se inicia el servidor y posteriormente los cuatro clientes de consola.

El servidor debe iniciarse primero. Cuando los cuatro jugadores se conectan, comienza la partida y el servidor asigna los turnos.

Durante el juego, los jugadores pueden realizar acciones como lanzar los dados, comprar propiedades, consultar su estado y terminar el turno.

Si se modifica el código, se deben detener las ejecuciones actuales, volver a compilar el proyecto y comenzar nuevamente la partida.

5. Demostración con 2 computadoras

Para realizar una demostración con dos computadoras, ambas deben estar conectadas a la misma red local. Una computadora funcionará como servidor y las demás se conectarán utilizando su dirección IP.

La Raspberry Pi Pico debe permanecer conectada a la computadora que funciona como servidor.

Antes de la demostración, se recomienda comprobar el Firewall de Windows y permitir la comunicación del programa en una red privada. También se debe realizar una prueba previa, ya que algunas redes institucionales pueden impedir la comunicación entre dispositivos. Como alternativa, se puede utilizar un hotspot.

6. Raspberry Pi Pico

La Raspberry Pi Pico se conecta mediante USB a la computadora que ejecuta el servidor. El puerto asignado por Windows se puede consultar desde Administrador de dispositivos → Puertos (COM y LPT).

Una vez identificado el puerto, se configura el servidor para utilizarlo. Durante cada turno, el sistema espera la señal del botón de la Pico durante un tiempo determinado. Si no recibe ninguna señal, utiliza el dado por software.

Desde el circuito se debe presionar el botón pulsador, el cual es el responsable de activar ambos displays y obtener los dos números y mandarlos al servidor. Una vez el sistema detecte que se debe cumplir un pago se activa el lector y se debe acercar la tarjeta o el llavero para completar el pago.

7. Finalización de la partida

Cuando la partida termina, el servidor informa a los clientes quién es el ganador. Este se determina según el patrimonio de cada jugador, considerando su saldo y sus propiedades. La partida también puede finalizar cuando queda un solo jugador activo.

Al finalizar, el sistema genera un reporte con las transacciones realizadas en la partida.

Una vez finalizada la partida, el sistema no permite realizar nuevas acciones.

Conexión del hardware

El módulo electrónico extiende el Monopoly con dos dispositivos físicos: un dado electrónico y un cajero con lector RFID, ambos controlados por una sola Raspberry Pi Pico W programada en MicroPython.

El dado utiliza dos displays de 7 segmentos y un botón. Cuando el servidor solicita un lanzamiento, el jugador presiona el botón y la Pico genera dos números entre 1 y 6, los muestra y los envía al servidor. Si no se presiona el botón en 15 segundos o no hay hardware conectado, el juego genera los valores por software para evitar que la partida se bloquee.

El cajero utiliza un lector RFID RC522 para identificar al jugador. El saldo se mantiene únicamente en el servidor y es modificado por el Banco, por lo que el hardware funciona solamente como medio de entrada y no toma decisiones sobre el estado del juego.

La Pico se conecta por USB a la computadora del servidor mediante un puerto serie virtual USB-CDC y utiliza un protocolo de texto propio, independiente del protocolo TCP del juego. Se utiliza USB para evitar problemas de comunicación en redes donde los dispositivos puedan estar aislados.

Como el dado y el lector RFID comparten el mismo puerto COM, el código en C# utiliza una única clase, PicoSerial, la cual es encargada de controlar el puerto y distribuir los mensajes entre ambos dispositivos.

Componentes	Descripción
Raspberry Pi Pico W	Microcontrolador con MicroPython.
2 displays de 7 segmentos de 0.50"	Cátodo común (modelo probado 5011AS, rojo).
14 resistencias de 220 Ω	Una por segmento (el rango seguro es 100–470 Ω).
Botón pulsador	Dispara el lanzamiento del dado.
LED de estado	Indicador del dado y del lector
Lector RFID RC522 y llaveros	Identifican al jugador.
Protoboard y cables jumper	Armado del circuito.

Display 1

Segmento	Pin	Segmento	Pin
A	GP28	E	GP8
B	GP27	F	GP9
C	GP2	G	GP10
D	GP3	Común (pines 3 y 8)	GND

Display 2

Segmento	Pin	Segmento	Pin
A	GP11	E	GP15
B	GP12	F	GP18
C	GP13	G	GP19
D	GP14	Común (pines 3 y 8)	GND

Botón, Led y módulo RFID

Componente	Pin de la Pico
Botón pulsador	GP16
LED de estado	GP20
RC522: SCK	GP6
RC522: MOSI	GP7
RC522: MISO	GP4
RC522: CS (SDA)	GP5
RC522: RST	GP22
RC522: alimentación	3.3V y GND